

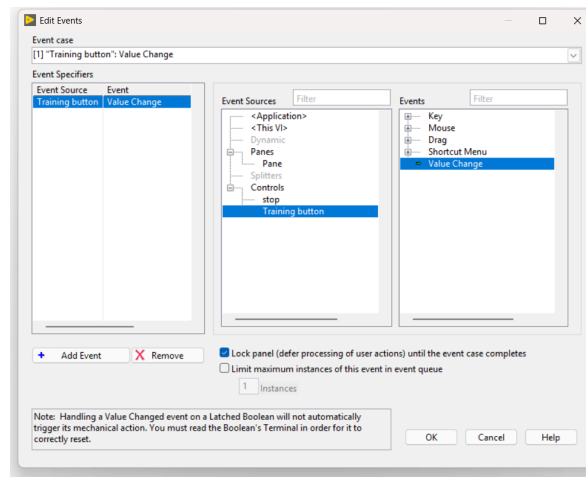
Guide for Training Model in LabVIEW

This guide explains how to program and wire a LabVIEW program with the objective of training AI models using the front panel in LabVIEW. To do this, a System Exec.vi block is used to run a 64-bit Python code that trains the model and saves data inside a specific csv and txt file that the LabVIEW will read to show results and important information.

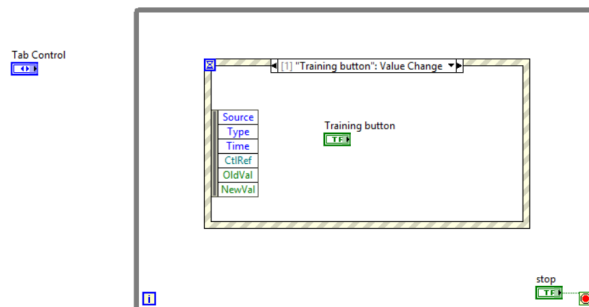
1. Setting up main loop

Before we start anything else, it would be wise to establish how the loop works. Our main program will be inside an Event Structure that's inside a While Loop, so you can start by creating that. You can also start by creating the stop button for the while loop, and an additional button that we will use to activate the event structure. Add an event case for when the training button value changes, as this is where most of our code will be.

(To add an event case, right click on the event structure and click on the “Add Event Case” option. A new menu will appear, choose the training button you created at the front end, select Value Change and click OK)



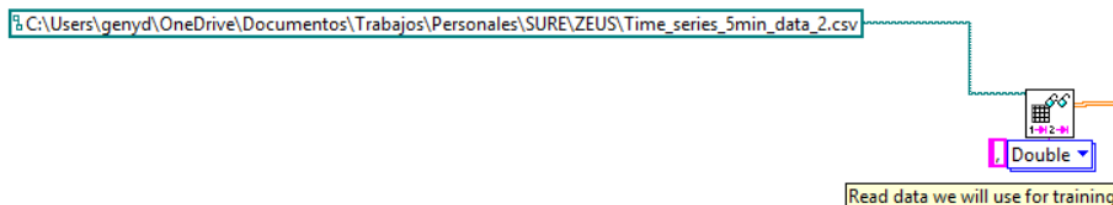
You now have your initial set up. Your back end should look like the image below (it is recommended that you add a Tab Control for better organization)



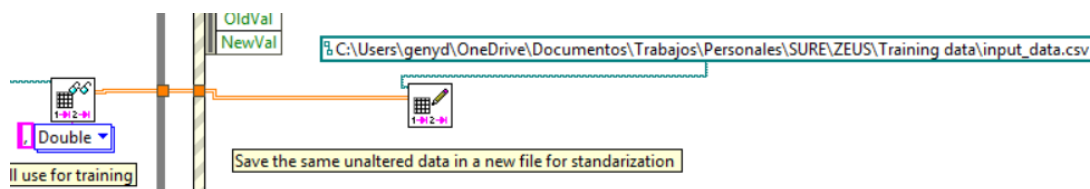
2. Setting up the data used for training

Since we are working with LabVIEW and Python, there needs to be standardized communication between both sides in order to avoid mistakes and have both codes run smoothly. For this purpose, the first thing to do inside our LabVIEW program is to open the csv file of the data used for training and save it unaltered in a new csv file with a standard name that the python code will be able to read.

This process is shown in the images below, in which we start by reading the original csv file outside our main loop using a Read Delimited Spreadsheet.vi block. Remember to wire the file path and the delimiter terminals, leaving a “,” inside the string constant wired to the delimiter terminal.



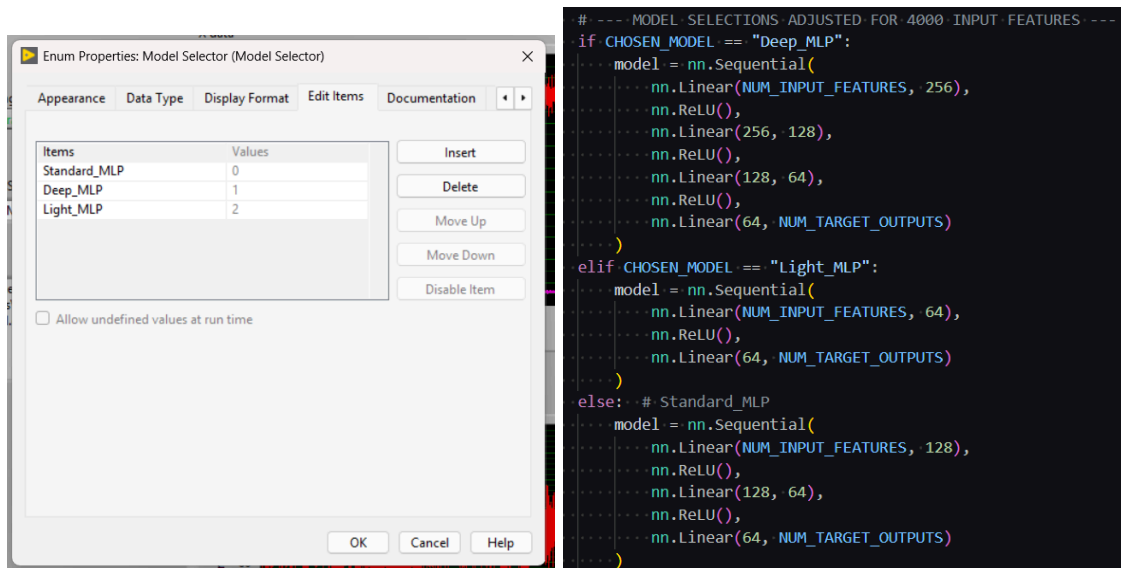
Then, we wire our all rows terminal into a Write Delimited Spreadsheet.vi block which is inside our event structure, in the “Training button” Value Change case. For this block we just need to wire the path in which we will save (in this case, a csv file named “input_data” which is the name that our python file expects to read)



3. Using the System Exec.vi block

The System Exec block needs only two wired terminals to work, the command line and the wait until completion? (T) terminals. To wire the latter, we only need to create a True constant and wire it. For the command line terminal, it is a little more complex. What this terminal does is send the string written into the terminal to windows, this the command we will send to our computer, and in that case, will run the python code. Because we want to be able to change the type or structure of the model we are training, this command line has to be able to change according to the model. To do this we need two things: a Format Into String block, and an Enum (created on the front panel). We will break down how to create and wire each:

- Enum: To create one, open the Tools Palette in the front panel and go Controls -> Ring & Enum -> Enum. You will then need to right click on the Enum and select properties. Select the “Edit Items” submenu. In this menu, you will insert the options you wish to have as model options. For example, this experiment has 3 options, “Standard_MLP”, “Light_MLP”, and “Deep_MLP”. It is important to take into account how you write each name of each model, since the Python model will need the exact correct name in order to interpret which model to train. The two images below show where the names are set in the LabVIEW Enum, and the way they are called exactly the same in the python code. The python code also shows the difference between each model for this example. Remember you can set the amount and names however you want, just be sure the match in LabVIEW and Python.



- Format Into string: This block will append the string from the Enum into the command line given to the System Exec block. You will wire into the input 1 terminal your Enum. And into your format string you need to wire a string constant that is the command line you will give windows to run your python script. The structure goes like this:

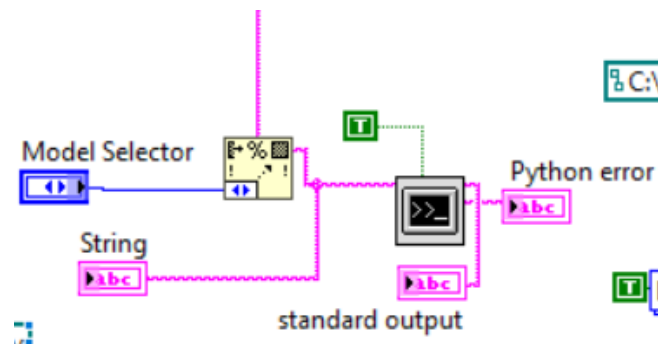
“Path for Python64-bit” “Path for Python script” -- model %s

For example, for this experiment the string constant is:

"C:\\Users\\genyd\\AppData\\Local\\Programs\\Python\\Python312\\python.exe"
 "C:\\Users\\genyd\\OneDrive\\Documentos\\Trabajos\\Personales\\SURE\\ZEUS\\Training data\\LabVIEW_training.py" --model %s

The word “-- model” is needed to indicate to the python code this is the part to decode in order to know which model to train, and the part “%s” indicates to the Format Into String where to put the input 1.

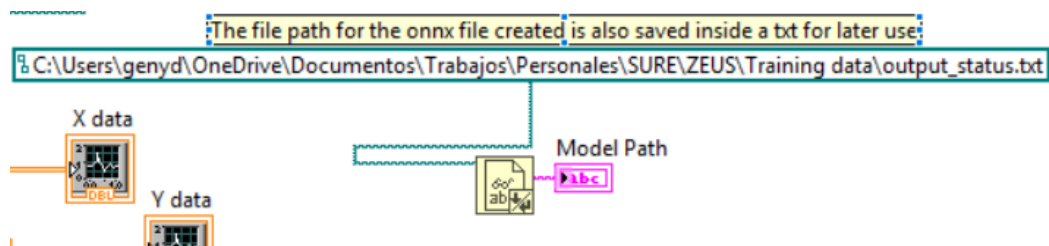
Finally, you wire the “resulting string” terminal into the “command line” terminal from the format into string to the System Exec block. Now you have your System Exec.vi fully configured, and you can wire an indicator to the command string wire and the standard output and error terminals. This is not necessary but very useful for debugging. Your blocks should look something like the image below.



4. Reading and showing results

The final step in this LabVIEW code is to read the csv and txt files created by the Python code we ran. To do this, we use a Read Delimited Spreadsheet.vi block again for the csv file, and a Read from Text File block for the txt.

- Read from Text File: You only need to wire the file path, and if wanted an indicator to the text terminal to show the path. The path for the txt file should be the same as the one where you have your python script, only changing it for the txt file.



- Read Delimited Spreadsheet.vi: As always, you need to wire the path of your csv file to the path terminal. This file should also be in the same directory as your python code. Wire the Transpose? (F) terminal to a True constant. To show the results you need to create 2 waveform graphs in your front panel. And to wire the

n error

all rows

The file

C:\Users\genyd\OneDrive\

Double

X data

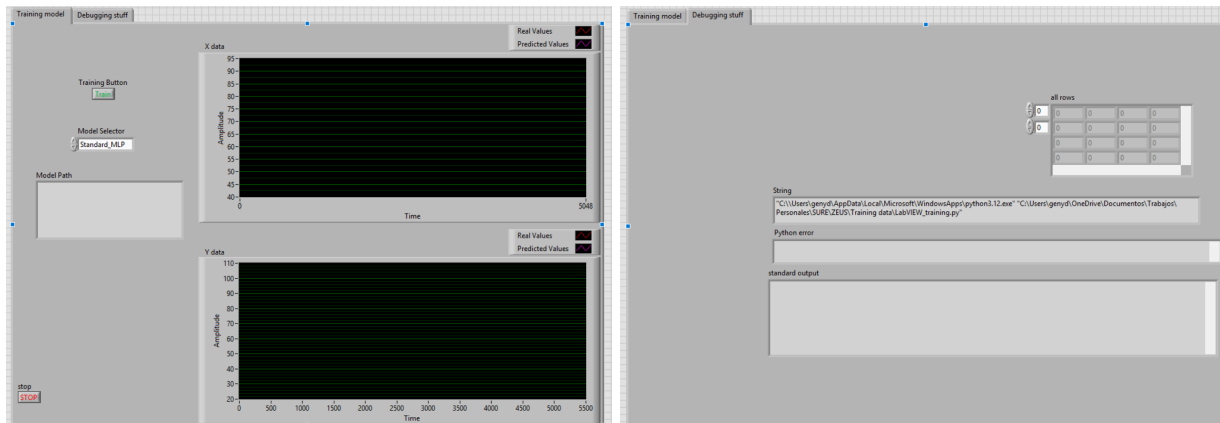
Y data

Bundle both X and Y real and predicted data for a single graph for each coordinate

The screenshot displays a complex Alteryx workflow designed for training a Linear Regression model. The process begins with two input tools: 'Training Button' (XLS) and 'Tab Control' (XLS). These inputs feed into a 'Read data we will use for training' (XLS) tool, which connects to a CSV file located at 'S:\Users\jerry\OneDrive\Documents\Tabcontrol\Personal\SURE\2015\Time series_Sema_data_2.csv'. The data is then processed by a 'Model Selector' (XLS) tool, which outputs to a 'standard output' (XLS) tool. This tool feeds into a 'Python' tool, which executes a command to run a Jupyter Notebook: 'C:\Users\jerry\AppData\Local\Programs\Python\Python32\python.exe -C:\Users\jerry\OneDrive\Documents\Tabcontrol\Personal\SURE\2015\Training data\LabVIEW training.py --model 1d'. The Python tool's output is processed by a 'Python error' tool, which then feeds into a 'Model Path' (XLS) tool. The final output is saved to a file: 'C:\Users\jerry\OneDrive\Documents\Tabcontrol\Personal\SURE\2015\Training data\output_model.csv'. The workflow also includes a 'Save the same unsorted data in a new file for standardization' (XLS) tool and a 'Read data we will use for training' (XLS) tool. The entire workflow is contained within a 'Training Button\' Value Change' (XLS) tool, which is connected to a 'stop' (XLS) tool at the end.

How to use this LabVIEW program

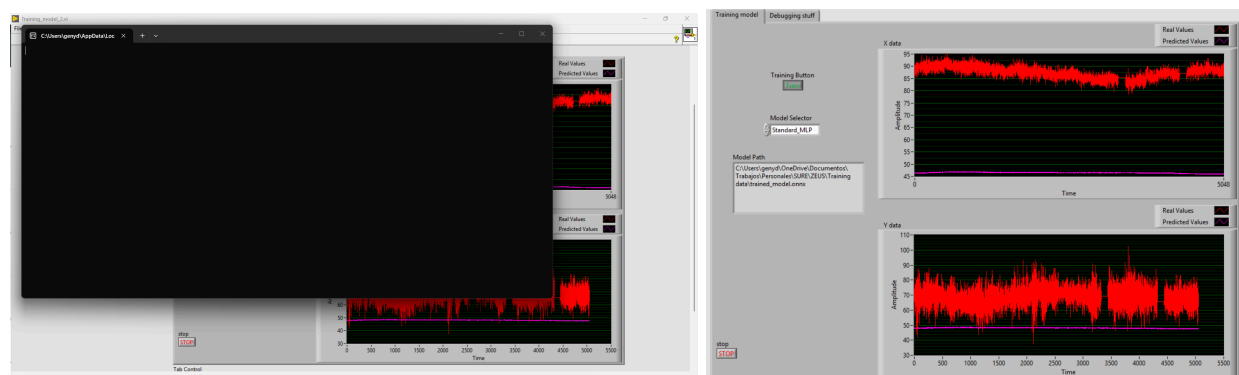
By the end, your front panel should look something like the images below, but you can arrange things as it is more comfortable to you.



To use the code, you simply need to run the program and follow the next steps:

1. Select in your Enum (Model Selector) the model you want to train
2. Click on the training button
3. Wait for the python code to run
4. Analyze results

When you click the training button, a new window will pop up and LabVIEW will freeze as long as the python code is running. When it finishes the waveform graphs show the predicted X and Y coordinates vs the real values. The images below show both the results after the code, and the opening of the window that runs the code. (The results in the example are bad, as it was only necessary for the code to work)



You will also have a second window where you (if you created the indicators) have information about the program. This window shows the data read from the csv created by the python file in the array All Rows. It also shows the Python warnings and errors as

well as any printed information in your code with the standard output indicator. Finally, it also shows the final command line sent to windows in the String indicator, so that you can check if you are sending the correct command line.

The screenshot shows a LabVIEW interface with two tabs: "Training model" and "Debugging stuff". The "Debugging stuff" tab is active, displaying a table of data, a string of the command line, a Python error message, and standard output logs.

all rows

0	87.922	85.675	89.596	90.514
0	46.4142	46.4083	46.3684	46.3855
	68.055	71.773	70.814	71.423
	48.0472	47.8262	47.951	47.9542

String

```
"C:\Users\genyd\AppData\Local\Programs\Python\Python312\python.exe" "C:\Users\genyd\OneDrive\Documentos\Trabajos\Personales\SURE\ZEUS\Training data\LabVIEW_training.py" --model Standard_MLP
```

Python error

```
W0713 10:38:37.022000 10008 site-packages\torch\onnx\internal\exporter\registration.py:107] torchvision is not installed. Skipping  
torchvision:nms
```

standard output

```
=====  
[INFO] TIMEFORECAST PIPELINE: Initializing Windowed Training...  
[INFO] Selected Architecture: Standard_MLP  
=====  
[INFO] Reading dataset...  
[DATA] Total raw rows detected: 27242  
[DATA] Generating sliding windows (Size: 2000 frames)...
```

Used Code for this example

```
import os
import sys
import argparse

# FORCE PYTHON TO USE UTF-8 ENCODING FOR ALL PRINTS
if hasattr(sys.stdout, 'reconfigure'):
    sys.stdout.reconfigure(encoding='utf-8')

# --- COMMAND LINE ARGUMENT PARSING ---
parser = argparse.ArgumentParser(description="LabVIEW Time-Forecast Training Pipeline")
parser.add_argument(
    '--model',
    type=str,
    default='Standard_MLP',
    choices=['Standard_MLP', 'Deep_MLP', 'Light_MLP'],
    help='Select the neural network architecture layout'
)
args = parser.parse_args()
CHOSEN_MODEL = args.model

print("\n=====")
print(f"[INFO] TIMEFORECAST PIPELINE: Initializing Windowed Training...")
print(f"[INFO] Selected Architecture: {CHOSEN_MODEL}")
print("=====")

import torch
import torch.nn as nn
import torch.optim as optim

# 1. Setup paths
WORKING_DIR = r"C:\Users\genyd\OneDrive\Documentos\Trabajos\Personales\SURE\ZEUS\Training data"
INPUT_FILE = os.path.join(WORKING_DIR, "input_data.csv")
OUTPUT_PREDICTIONS = os.path.join(WORKING_DIR, "output_loss.csv") # Kept name intact for LabVIEW path
OUTPUT_STATUS = os.path.join(WORKING_DIR, "output_status.txt")
ONNX_FILE = os.path.join(WORKING_DIR, "trained_model.onnx")

WINDOW_SIZE = 2000

def main():
    if not os.path.exists(INPUT_FILE):
        print(f"[ERROR] Cannot find data at {INPUT_FILE}")
        sys.exit(1)

    print("[INFO] Reading dataset...")
    raw_data = []
```



```

with open(INPUT_FILE, "r") as f:
    for line in f:
        if line.strip():
            row = [float(val) for val in line.strip().split('\t')]
            raw_data.append(row)

data_tensor = torch.tensor(raw_data, dtype=torch.float32)
total_rows = data_tensor.shape[0]

print(f"[DATA] Total raw rows detected: {total_rows}")
if total_rows <= WINDOW_SIZE:
    print(f"[ERROR] Dataset length ({total_rows}) must be greater than window size ({WINDOW_SIZE}).")
    sys.exit(1)

# --- VECTORIZED SLIDING WINDOW GENERATION ---
print(f"[DATA] Generating sliding windows (Size: {WINDOW_SIZE} frames)...")
num_samples = total_rows - WINDOW_SIZE

# Pre-allocate tensors for speed
X = torch.zeros((num_samples, WINDOW_SIZE * 2), dtype=torch.float32)
Y = torch.zeros((num_samples, 2), dtype=torch.float32)

for i in range(num_samples):
    X[i] = data_tensor[i : i + WINDOW_SIZE].flatten() # Flattens 2000x2 into 4000
    Y[i] = data_tensor[i + WINDOW_SIZE]               # Next target point [X, Y]

NUM_INPUT_FEATURES = WINDOW_SIZE * 2 # 4000
NUM_TARGET_OUTPUTS = 2               # Predicting both X and Y

# --- CHRONOLOGICAL 80/20 SPLIT (Standard for Time Series) ---
train_size = int(0.8 * num_samples)

X_train, Y_train = X[:train_size], Y[:train_size]
X_test, Y_test = X[train_size:], Y[train_size:]

print(f"[DATA] Train Split: {X_train.shape[0]} windows | Test Split: {X_test.shape[0]} windows")

# --- MODEL SELECTIONS ADJUSTED FOR 4000 INPUT FEATURES ---
if CHOSEN_MODEL == "Deep_MLP":
    model = nn.Sequential(
        nn.Linear(NUM_INPUT_FEATURES, 256),
        nn.ReLU(),
        nn.Linear(256, 128),
        nn.ReLU(),
        nn.Linear(128, 64),
        nn.ReLU(),
        nn.Linear(64, NUM_TARGET_OUTPUTS)
    )

```

```

elif CHOSEN_MODEL == "Light_MLP":
    model = nn.Sequential(
        nn.Linear(NUM_INPUT_FEATURES, 64),
        nn.ReLU(),
        nn.Linear(64, NUM_TARGET_OUTPUTS)
    )
else: # Standard_MLP
    model = nn.Sequential(
        nn.Linear(NUM_INPUT_FEATURES, 128),
        nn.ReLU(),
        nn.Linear(128, 64),
        nn.ReLU(),
        nn.Linear(64, NUM_TARGET_OUTPUTS)
    )

criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.005)

epochs = 20
print(f"[TRAIN] Training {CHOSEN_MODEL} forecast engine for {epochs} epochs...")

model.train()
for epoch in range(epochs):
    optimizer.zero_grad()
    predictions = model(X_train)
    loss = criterion(predictions, Y_train)
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 5 == 0 or epoch == 0:
        print(f" Epoch [{epoch+1}/{epochs}] | Train MSE Loss: {loss.item():.6f}")

# Evaluation
print("[EVAL] Running predictions on test timeline...")
model.eval()
with torch.no_grad():
    test_preds = model(X_test)

# Export to ONNX
print("[SAVE] Exporting time-series engine to ONNX format...")
dummy_input = torch.randn(1, NUM_INPUT_FEATURES)
torch.onnx.export(
    model,
    dummy_input,
    ONNX_FILE,
    export_params=True,
    opset_version=18,
    do_constant_folding=True,
    input_names=['input_trajectory'],

```

```

    output_names=['predicted_xy']
)

# --- SAVE 4-COLUMN TRACKING DATA FOR LABVIEW ---
print("[SAVE] Exporting X and Y parallel track data...")
# Format: X_Real \t X_Pred \t Y_Real \t Y_Pred
with open(OUTPUT_PREDICTIONS, "w") as f:
    Y_test_list = Y_test.tolist()
    test_preds_list = test_preds.tolist()
    for i in range(len(Y_test_list)):
        f.write(f"{Y_test_list[i][0]}\t{test_preds_list[i][0]}\t{Y_test_list[i][1]}\t{test_preds_list[i][1]}\n")

# Write status and ONNX path
status_msg = f"{ONNX_FILE}"
with open(OUTPUT_STATUS, "w") as f:
    f.write(status_msg)

print("[SUCCESS] Time-forecast training finished without errors!")

if __name__ == "__main__":
    main()

```